МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 6

Тема: Структурные паттерны: Заместитель

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы структурного паттерна проектирования «Заместитель» (Proxy) и реализовать прикладную задачу «Графический пароль» с использованием данного паттерна.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать структурный паттерн «Заместитель».

В соответствии с заданием был выбран паттерн «Заместитель» (Proxy).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу паттерна «Заместитель»: прокси контролирует доступ к реальному сервису пароля, добавляет логирование, ленивую инициализацию и защиту от подбора.

1. Описание паттерна «Заместитель»

Паттерн «Заместитель» (Proxy) - это структурный паттерн проектирования, который предоставляет объект-заместитель, контролирующий доступ к другому объекту. Заместитель имеет тот же интерфейс, что и реальный объект, поэтому клиент работает с ним так же, как с оригиналом, не замечая подмены.

Данный паттерн применяется в ситуациях, когда нужно контролировать доступ к объекту, добавить ленивую инициализацию, кэширование, логирование или защиту без изменения кода самого объекта.

В разработанном приложении паттерн «Заместитель» используется для управления доступом к подсистеме графического пароля. В программе определены следующие роли:

- PasswordService - общий интерфейс (Subject), определяющий методы has_password(), create_password(sequence), verify_password(sequence) и reset_password();

- PasswordManager - реальный сервис (RealSubject), реализующий Singleton и SHA-256 хэширование;

- PasswordManagerProxy - заместитель (Proxy), реализующий тот же интерфейс.

Заместитель добавляет следующее поведение:

- Ленивая инициализация - реальный PasswordManager создаётся только при первом обращении, а не при старте приложения;

- Защита от подбора - после 5 неверных попыток вход блокируется на 30 секунд;

- Логирование - все операции записываются в лог с меткой времени;

- Счётчик неудачных попыток - доступен через get_failed_attempts().

Таким образом, клиентский код (AppWindow) работает с PasswordService, не зная, что за ним стоит заместитель с дополнительной логикой.

2. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования:

- длина последовательности от 4 до 9 ячеек;

- одна и та же ячейка не может быть выбрана повторно;

- созданный пароль сохраняется в виде SHA-256 хэша.

Приложение содержит два режима: «Создание пароля» и «Вход».

Дополнительно реализованы:

- кнопка «Показать лог прокси» для просмотра журнала операций;

- счётчик неудачных попыток под сеткой;

- автоматическая блокировка после 5 неверных попыток на 30 секунд;

- кнопка «Сбросить сохранённый пароль».

Все действия с паролем выполняются через PasswordManagerProxy.

3. Диаграмма классов

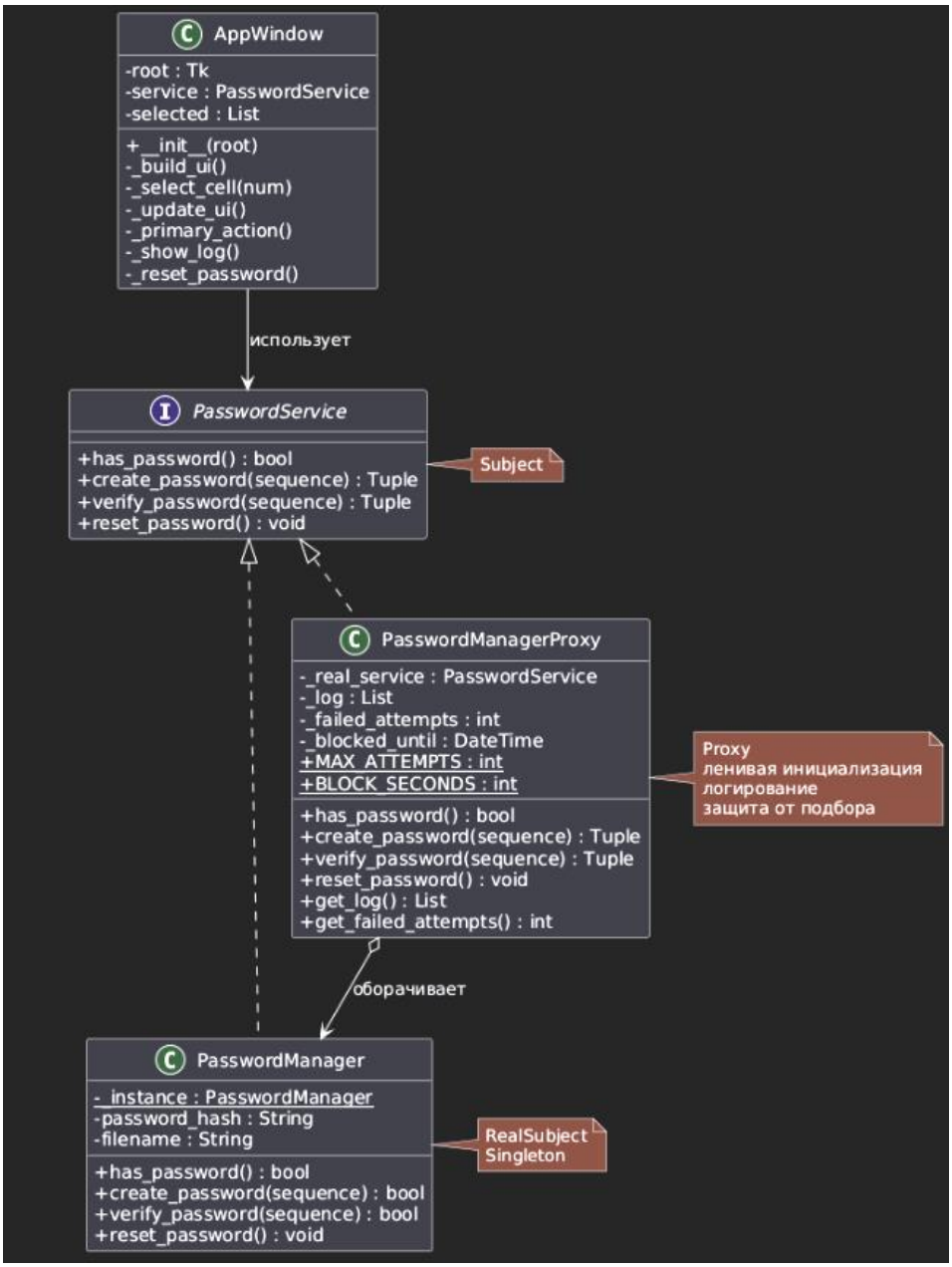


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

- PasswordService - общий интерфейс (Subject);

- PasswordManager - реальный сервис (RealSubject, Singleton);

- PasswordManagerProxy - заместитель (Proxy);

- AppWindow - клиентский класс.

4. Реализация паттерна «Заместитель»

Общий интерфейс:

class PasswordService(ABC):

```
    @abstractmethod
    def has_password(self):
        pass

    @abstractmethod
    def create_password(self, sequence):
        pass

    @abstractmethod
    def verify_password(self, sequence):
        pass

    @abstractmethod
    def reset_password(self):
        pass
```

Реальный сервис PasswordManager реализует этот интерфейс и Singleton (см. лабу 1).

Заместитель:

```
class PasswordManagerProxy(PasswordService):
    MAX_ATTEMPTS = 5
    BLOCK_SECONDS = 30

    def __init__(self):
        self._real_service = None
        self._log = []
        self._failed_attempts = 0
        self._blocked_until = None

    def _get_service(self):
```

```
        if self._real_service is None:
            self._real_service = PasswordManager()
        return self._real_service


    def verify_password(self, sequence):
        if self._is_blocked():
            return False, "Доступ заблокирован"
        if not validate_sequence(sequence):
            return False, "Некорректная последовательность."
        result = self._get_service().verify_password(sequence)
        if result:
            self._failed_attempts = 0
            return True, "Пароль верный."
        self._failed_attempts += 1
        if self._failed_attempts >= self.MAX_ATTEMPTS:
            self._blocked_until = datetime.now() +
    timedelta(seconds=self.BLOCK_SECONDS)
            return False, f"Блокировка на {self.BLOCK_SECONDS} секунд."
        return False, f"Неверный пароль. Попытка
    {self._failed_attempts}/{self.MAX_ATTEMPTS}."
```

Клиент AppWindow работает только через self.service = PasswordManagerProxy(), вызывая методы has_password(), create_password(), verify_password() и reset_password().

Логирование и защита от подбора реализованы внутри заместителя, а не в клиенте и не в реальном сервисе - это и есть суть паттерна.

5. Результаты работы программы

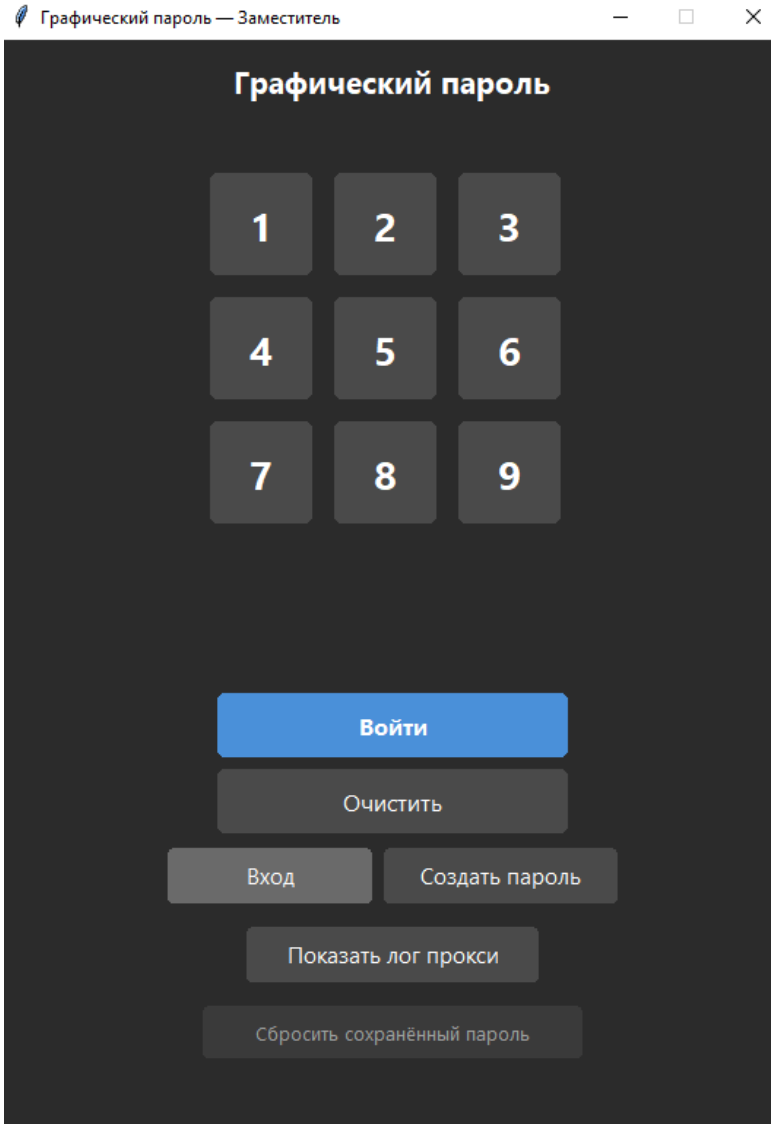


Рисунок 2 - Главное окно приложения

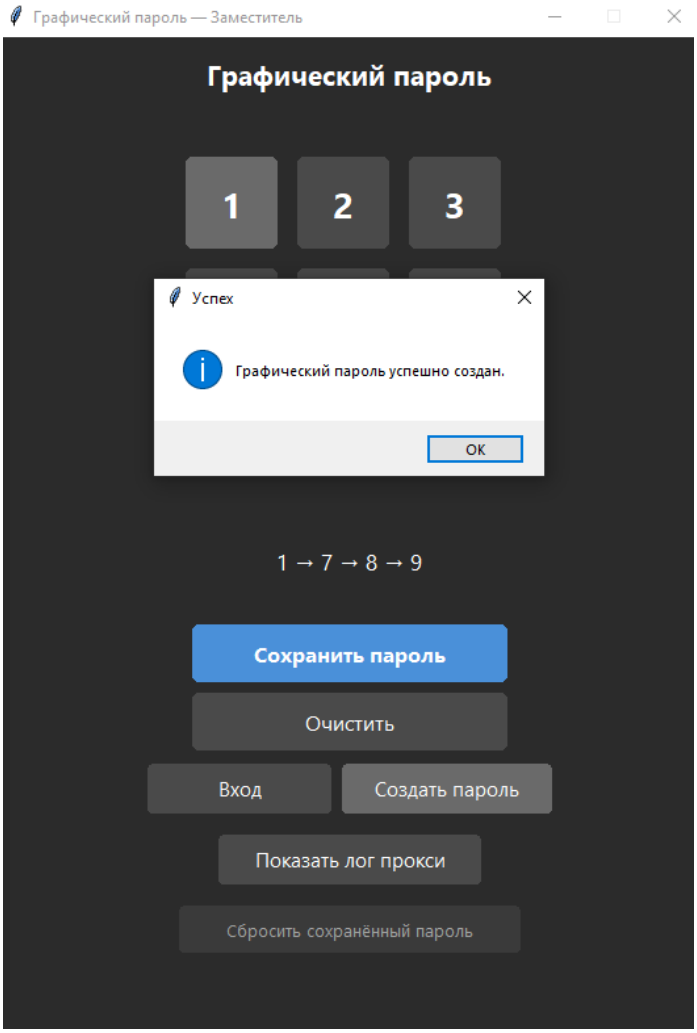


Рисунок 3 - Создание пароля

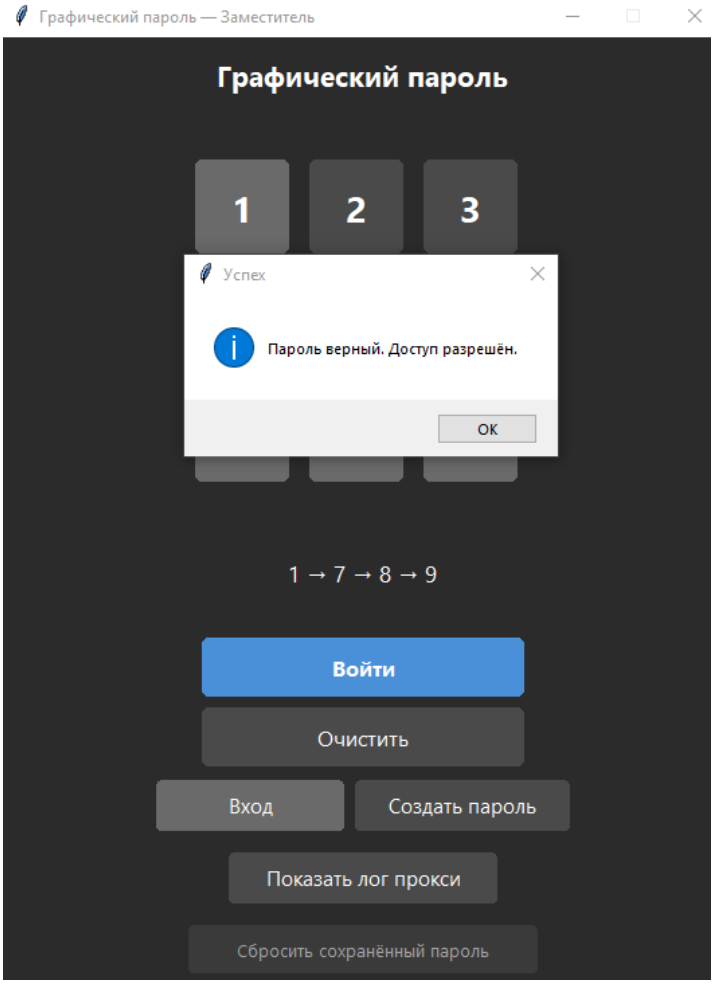


Рисунок 4 - Успешная проверка пароля

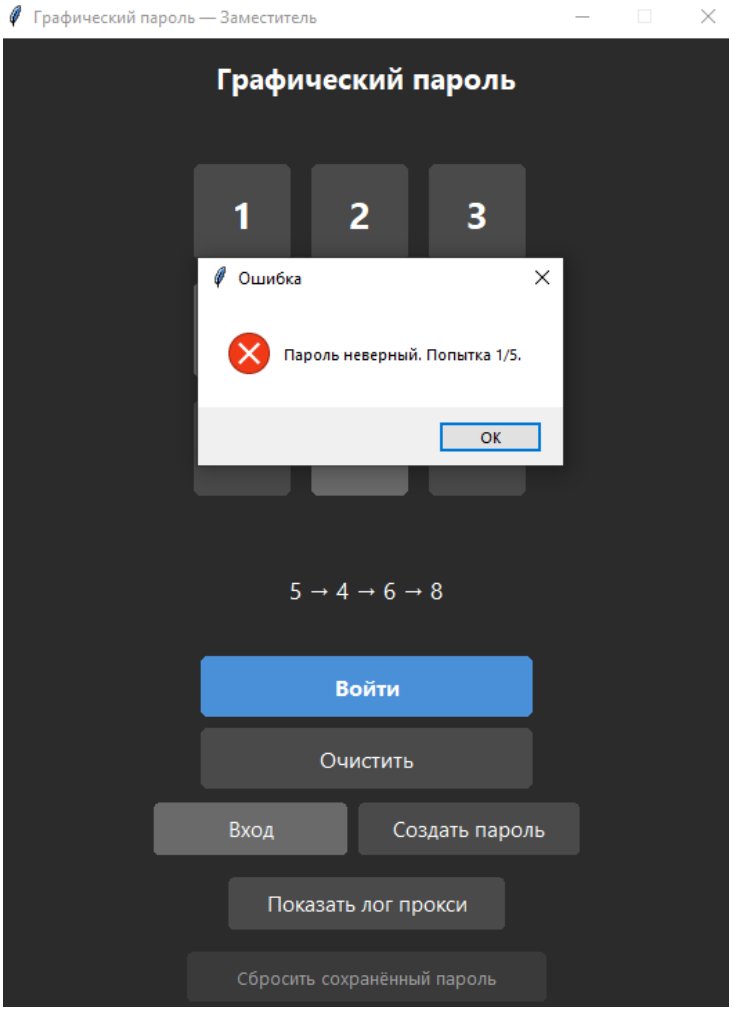


Рисунок 5 - Неверный пароль (счётчик попыток)

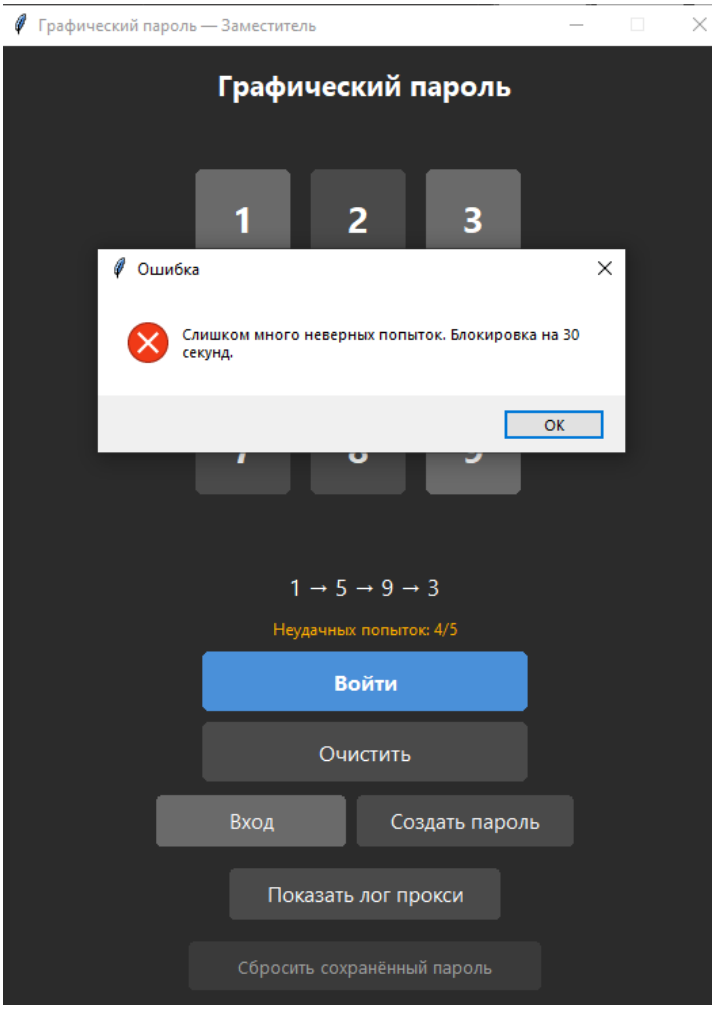


Рисунок 6 - Блокировка после 5 неверных попыток

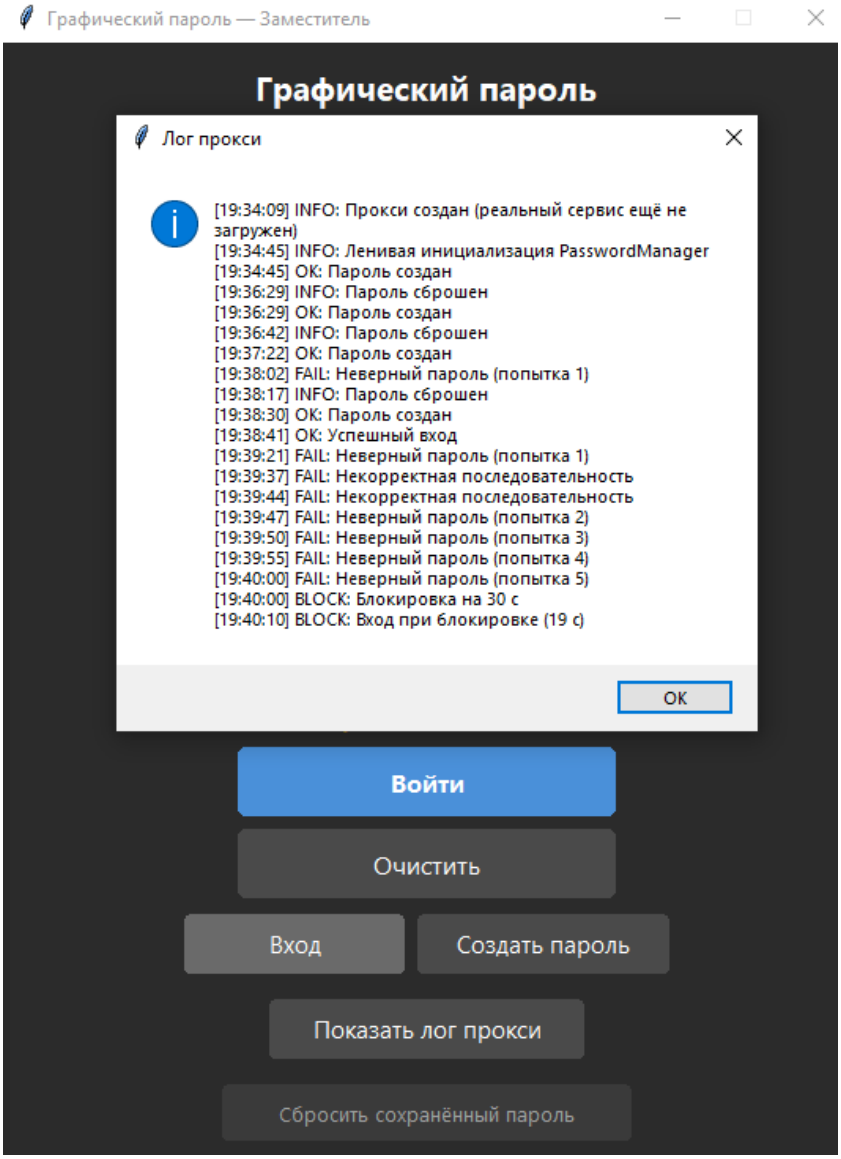


Рисунок 7 - Лог прокси (ленивая инициализация, логирование, блокировка)

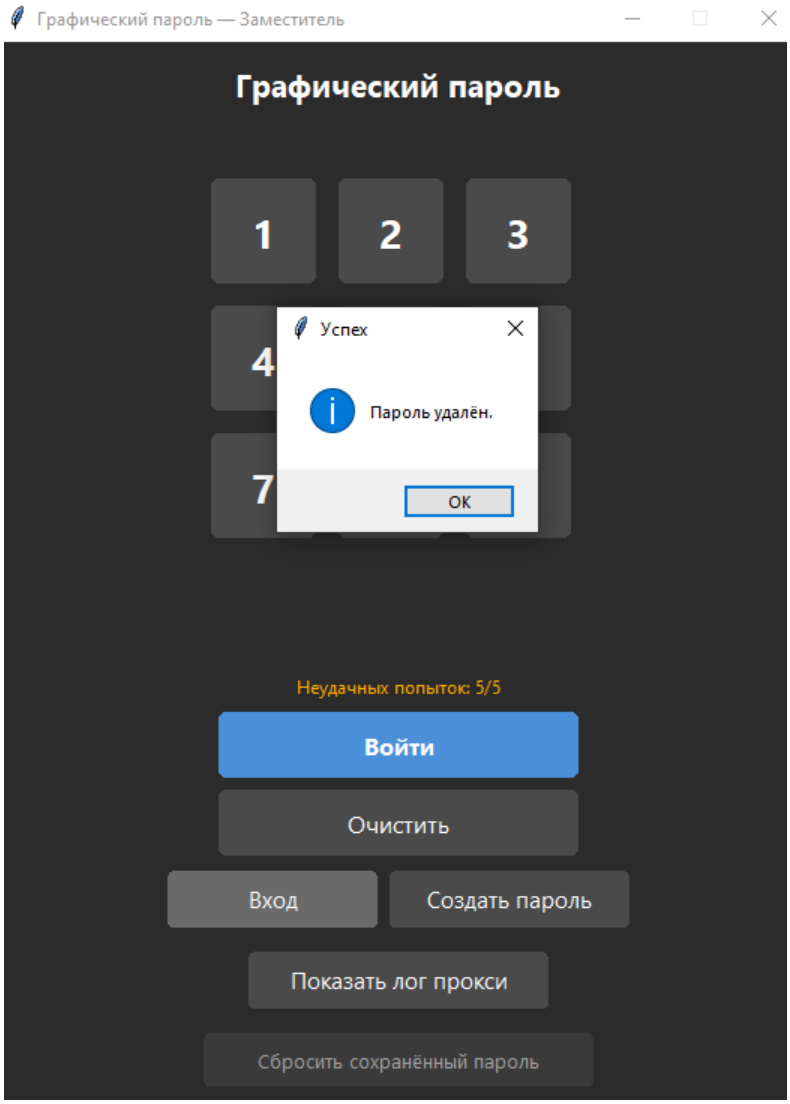


Рисунок 8 - Сброс сохранённого пароля

6. Исходный код

Основные элементы программы:

- PasswordService - общий интерфейс (Subject);
- PasswordManager - реальный сервис (RealSubject, Singleton);
- PasswordManagerProxy - заместитель (Proxy) с ленивой инициализацией, логированием и защитой;
- AppWindow - основной класс интерфейса;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;
- validate_sequence(), hash_sequence() - вспомогательные функции;
- graphical_password.json - файл хранения хэша пароля.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Был изучен и реализован структурный паттерн проектирования «Заместитель» (Proxy). Паттерн применён через класс PasswordManagerProxy, который реализует тот же интерфейс PasswordService, что и реальный PasswordManager, и добавляет ленивую инициализацию, логирование и защиту от подбора (блокировка после 5 неверных попыток на 30 секунд). Клиентский код (AppWindow) работает с общим интерфейсом PasswordService, не зная о наличии заместителя. В результате получена работоспособная программа, демонстрирующая практическое применение структурного паттерна «Заместитель» при разработке прикладного программного обеспечения.